

polars-bio w trybie rozproszonym: Ballista i Sail

Podsumowanie techniczne prac — praca magisterska

Miłosz Kowalewski

26 sierpnia 2026

1. Cel i założenia projektu

Temat pracy magisterskiej: porównanie udostępnianych na otwartych licencjach rozproszonych silników zapytań pod kątem analiz danych genomicznych, na przykładzie integracji biblioteki **polars-bio** z dwoma silnikami: **Apache Ballista** oraz **LakeSail/Sail**.

Projekt realizuje następujące założenia:

1. Cel: ogólny mechanizm dodawania funkcji użytkownika (UDF/UDTF/UDA) na poziomie DataFusion, wpinany do silnika rozproszonego jako *runtime extension*, **bez forkowania** samego silnika.
2. Docelowo porównanie **dwóch** silników — Ballista i Sail — z pełną implementacją w obu.
3. Aspekt naukowy pracy to wydajna implementacja rozproszona, dobór algorytmów oraz benchmarki.
4. Docelowo zmiany mają trafić do samego polars-bio (nowa reguła optymalizatora: lokalna vs. rozproszona ścieżka wykonania) — nie mają pozostać zbiorem niezależnych skryptów obok biblioteki. Ten krok potraktowano jako opcjonalny i odłożony w czasie (Faza F, patrz niżej), ponieważ wymaga osobnej decyzji dotyczącej samej biblioteki.
5. Zmiana **planu zapytania** (repartycja danych według chromosomu) jest konieczna w obu silnikach; sama serializacja danych nie stanowi problemu, ponieważ oba silniki operują na wspólnym standardzie Apache Arrow.
6. UDF/UDTF ma faktycznie wywoływać silnik polars-bio, a nie reimplementować algorytm od zera.
7. Punkty odniesienia SeQuiLa i Comet odrzucono jako nieadekwatne.

Praca prowadzona jest w cyklu: implementacja kroku → test poprawności względem wyroczni referencyjnej (`pb.overlap()` i pozostałe funkcje polars-bio uruchomione lokalnie) → poprawki → ponowny test.

2. Architektura docelowa

Z punktu widzenia użytkownika, punktem wejścia pozostaje koncepcyjnie polars-bio (`pb.overlap(df_a, df_b)`), a silnik rozproszony jest wybierany pod spodem backendem. Ponieważ Ballista i Sail należą do różnych rodzin technologicznych, zastosowano dwa różne wzorce integracji.

Ważne ograniczenie zakresu przyjęte na obecnym etapie: przez całą fazę weryfikacji wykonalności (Fazy 0–D) **nie modyfikuje się kodu źródłowego polars-bio**. Cała logika wyboru silnika żyje jako osobny moduł w repozytorium pracy magisterskiej i woła wyłącznie publiczne, niezmiennicze funkcje z zainstalowanego pakietu polars-bio. Właściwa integracja wewnątrz biblioteki to osobny, opcjonalny krok (Faza F).

Ballista — integracja bezpośrednia (jedna rodzina DataFusion)

```
distributed_overlap(df_a, df_b, engine="ballista") [kod projektu, poza polars-bio]
-> buduje plan zapytania z operatorem overlap() z datafusion-bio-function-ranges
-> wysyła plan do klastra Ballista (scheduler + executory)
-> scheduler repartycjonuje dane wg chromosomu
-> każdy executor liczy lokalny overlap (COITrees) na swojej partycji
-> wyniki wracają, złożone w jeden DataFrame
```

Sail — pętla przez UDTF (inna rodzina: Spark Connect)

```
distributed_overlap(df_a, df_b, engine="sail") [kod projektu, poza polars-bio]
-> łączy się z klastrem Sail przez protokół Spark Connect
-> Sail repartycjonuje dane wg chromosomu (standardowy groupBy)
-> zarejestrowany UDTF, wywoływany na każdej partycji, WOŁA Z POWROTEM pb.overlap()
    (niezmieniona, publiczna funkcja z zainstalowanego pakietu polars-bio)
-> wynik wraca przez Saila do kodu projektu -> do użytkownika
```

Dla Saila „UDTF woła polars-bio” i „kod projektu woła Saila” to dwa końce tej samej pętli, a nie konkurencyjne podejścia — i żadne z nich nie modyfikuje źródeł polars-bio.

3. Przebieg prac (Fazy 0-C)

Faza 0 — porządki

Zaktualizowano `analiza_literatury.md` i `architektura_draft.md`, usuwając nieaktualny wniosek „Sail odrzucony” (oparty na wcześniejszym, błędnym założeniu) i opisując aktualny stan: Sail jako kandydat do integracji przez UDTF, z mechanizmem `SailExtension/FFI` dla UDF/optimizer rules dopiero projektowanym po stronie zespołu Sail (dyskusja `sail#2001`), niezaimplementowanym w chwili pisania pracy. Zespół SedonaDB prowadzi analogiczną integrację (spatial join, koncepcyjnie bliską genomicznemu overlap) na forku `james-willis/sail` — potraktowane jako materiał referencyjny/literaturowy, potwierdzający że natywna integracja na poziomie planu zapytania w Sailu obecnie wymaga forka. Bez forka Sail oferuje dziś jedynie UDTF (PR #1519 w `pysail`) — i to przyjęto jako w pełni legalny, docelowy (nie tymczasowy) wynik dla Saila w tej pracy.

Faza A — Ballista, wykonalność lokalna i rozproszona

Katalog roboczy: `ballista_genomics/`.

Krok 1-2. Kluczowe odkrycie na starcie: prawdziwym silnikiem algorytmicznym za `polars-bio` jest zewnętrzny `crate datafusion-bio-function-ranges` (`github.com/biodatageeks/datafusion-bio-functions`, Apache-2.0) — eksponuje operacje (`overlap`, `merge`, `nearest`, `coverage`, `subtract`, `cluster`, `complement`) jako gotowe funkcje tabelowe `DataFusion`, z wyborem algorytmu (`COITrees`, `IntervalTree`, `Lapper`, `SuperIntervals`). Wcześniejszy prototyp implementował naiwny predykat `overlap` $O(n \cdot m)$ w UDF-ie, bez użycia tego silnika w ogóle. Dodano `crate` jako zależność (wymagało to podniesienia wersji `datafusion/ballista` do 53.0.0, bo `crate` pina tę wersję) i zweryfikowano lokalnie (bez Ballisty) — wynik: 8/8 par identycznych z `pb.overlap()`, tym razem faktycznie liczonych przez `COITrees`.

Krok 3. Przepisano `main.rs` na `SessionContext::standalone_with_state()` (klaster `inproc`: `scheduler` + `executor`). Zapytanie z `overlap(...)` zakończyło się oczekiwanym błędem `LogicalExtensionCodec is not provided` — `DataFusion/Ballista` nie potrafi domyślnie zserializować niestandardowego węzła planu logicznego, żeby przesłać go od klienta do schedulera.

Krok 4 — główny cel Fazy A. Zaimplementowano własny `LogicalExtensionCodec` (`DistOverlapProvider`, delegujący do prawdziwego `OverlapProvider`; kodek serializuje jedynie parametry konstruktora, bez protobuf, ok. 50 linii). Wynik: **8/8 par identycznych z baseline, na prawdziwym klastrze Ballista** (scheduler + executor, rzeczywisty shuffle sieciowy widoczny w planie jako `ShuffleReaderExec`). Mechanizm wstrzyknięcia kodeka okazał się prostszy niż przewidywano: wystarczy `SessionConfigExt::with_ballista_logical_extension_codec(...)` na `SessionConfig` przed zbudowaniem sesji (re-eksportowane z `ballista::prelude`) — nie trzeba omijać `standalone_with_state()` ani dodawać osobnych zależności `ballista-core/-scheduler/-executor`. Ten etap miał jednak istotny kompromis: plan fizyczny na executorze spadał do zwykłego `HashJoinExec` zamiast korzystać z `COITrees` (patrz Faza A.5 niżej — kompromis ten został później naprawiony).

Faza A.5 — COITrees w pełni rozproszonym overlap (opisana szerzej w sekcji 4.1)

Faza B — Sail, wykonalność lokalna

Plik: `sail_overlap_udtf.py`. Zastąpiono `applyInPandas` prawdziwym UDTF (PR #1519 w `pysail`): klasa z metodą `eval()`, rejestrowana przez `spark.udtf.register(...)`, wywoływana przez `LATERAL overlap_udtf(...)`. Wewnątrz `eval()` nadal wołane jest niezmiennicze `pb.overlap()`. Wynik: **8/8 par, identyczne z lokalnym pb.overlap()**.

Po drodze natrafiono na dwa istotne ograniczenia `pysail` 0.5.3:

1. **Brak wsparcia dla argumentów typu TABLE w UDTF** — wypróbowano trzy warianty składni (`TABLE(...)` `PARTITION BY` w SQL, to samo przez `DataFrame API/TableArg`, argument `TABLE` bez opcji) — każdy kończy się innym błędem parsera/silnika. PR #1519 dodał rejestrację UDTF i argumenty **skalarne**, ale nie `TABLE`.
2. **Gubienie/duplikowanie wierszy przy więcej niż jednej partycji fizycznej** (zweryfikowane: 2 grupy na 2 partycjach → jedna zgubiona całkowicie, druga zdublowana dwukrotnie). Obejściem było wymuszenie jednej partycji (`.repartition(1)`) przed wywołaniem UDTF — naprawiało poprawność w 100%, ale kosztem całej równoległości.

Uwaga: pierwotna atrybucja tego objawu do błędu Saila okazała się nieprawidłowa. Faza H wykazała, że przyczyna leży po stronie `polars-bio` — szczegóły i dowody w sekcji 4.4.

Finalne, działające podejście: `groupBy("chrom").agg(collect_list(struct(...))) → .repartition(1)` (workaround na błąd 2) → `LATERAL overlap_udtf(chrom, rows_a, rows_b)`.

Faza C — rozszerzenie na pozostałe operacje

Sprawdzono kod źródłowy wszystkich pozostałych operacji w `datafusion-bio-function-ranges`: w przeciwieństwie do `OverlapProvider` (deleguje przez `session.sql()`), wszystkie pozostałe (`MergeProvider`, `NearestProvider`, `SubtractProvider`, `CountOverlapsProvider/coverage`, `ComplementProvider`, `ClusterProvider`) budują własne, **prywatne**, niestandardowe struktury `*Exec` bezpośrednio w `scan()`. Oznacza to, że żadna z tych operacji nie ma tej samej „furtki”, która pozwoliła osiągnąć pełną dystrybucję `overlap` w Ballistrze — wymagałoby to `PhysicalExtensionCodec` dla prywatnych typów (współpraca z autorami `crate’a` albo reimplementacja od zera). Zdecydowano: dla Fazy C testujemy tylko poziom lokalny w Ballistrze oraz `Sail` (UDTF) — pełna dystrybucja w Ballistrze pozostaje osiągnięciem specyficznym dla `overlap` (do czasu Fazy A.5, patrz niżej).

Status końcowy wszystkich pięciu operacji z pierwotnego zakresu (`overlap`, `merge`, `nearest`, `coverage`, `subtract`) — każda zweryfikowana na obu silnikach względem wyroczni `pb.*()`:

Operacja	Ballista (lokalnie)	Ballista (rozproszone)	Sail (UDTF)	Znalezisko
overlap	TAK	TAK (COITrees, Faza A.5)	TAK	—
merge	TAK	początkowo lokalnie; rozproszone od Fazy H	TAK	—
nearest	TAK	początkowo lokalnie; rozproszone od Fazy H	TAK	odwrócona orientacja względem <code>pb.nearest()</code> (korekta z 01.10.2026, punkt 6)
coverage	TAK	początkowo lokalnie; rozproszone od Fazy H	TAK	odwrócona konwencja argumentów względem <code>pb.coverage()</code>
subtract	TAK	początkowo lokalnie; rozproszone od Fazy H	TAK	—

Pełny pakiet testów: `tests/test*_correctness.py` (5 plików), łącznie z `test_oracle_sanity.py` i `test_ballista_overlap.py` — 18 testów przechodzących.

Poza pierwotnym zakresem pięciu operacji pozostają `cluster` i `complement`, również dostępne w `datafusion-bio-function-ranges` — do ustalenia, czy wchodzą w zakres pracy.

4. Sesja bieżąca: dogłębna weryfikacja wykonalności pełnej dystrybucji

Po zakończeniu Fazy C postawiono trzy pytania weryfikujące: czy wszystkie elementy planu są zrealizowane, czy da się w rozproszonej Ballistrze użyć `COITrees`, i czy `Sail` naprawdę działa w sposób rozproszony z serializowanym planem zapytania. W odpowiedzi przeprowadzono dwa niezależne dochodzenia opisane niżej.

4.1 Część 1 — `COITrees` w pełni rozproszonym `overlap` na Ballistrze

Problem. Węzeł planu fizycznego odpowiedzialny za wykonanie `overlap` przez `COITrees` to `IntervalJoinExec`, tworzony przez `IntervalJoinPhysicalOptimizationRule` z `datafusion-bio-function-ranges`. Ma on publiczny konstruktor `try_new()` i większość getterów, ale wymaga argumentu typu `ColIntervals` — struktury zdefiniowanej w module, który **nie jest publiczny** (mod `intervals` zamiast `pub mod intervals`). W Rust oznacza to, że typ jest nieosiągalny spoza `crate'a`, mimo że sama struktura ma widoczność `pub`.

Rozwiązanie. Zamiast rezygnować lub reimplementować algorytm od zera, `crate datafusion-bio-function-ranges` został **zvendorowany lokalnie** (`ballista_genomics/vendor/datafusion-bio-function-ranges/`) z

jednoliniową łatką widoczności (`mod intervals` → `pub mod intervals`, plus wygodny re-eksport). Jest to udokumentowane w `vendor/PATCH.md` jako **poprawka w pomocniczej bibliotece, nie fork Ballisty ani Saila** — zmiana czysto kosmetyczna (widoczność modułu), gotowa do zgłoszenia jako mały PR upstream.

Zaimplementowano `IntervalJoinPhysicalCodec: PhysicalExtensionCodec` (`ballista_genomics/src/physical_codec.rs` ok. 300 linii), który: - serializuje wyrażenia filtra przez istniejące funkcje `datafusion_proto` (`serialize_physical_expr/parse_physical_expr`), zamiast pisać własny format binarny dla wyrażeń, - odtwarza schemat pośredni filtra (`build_filter_schema`) oraz `ColIntervals` (przez zvendorowaną, teraz publiczną funkcję `parse_intervals`) po stronie odbiorcy zamiast serializować je wprost, - koduje ręcznie tylko proste, skończone enumeracje (`JoinSide`, `JoinType`, `PartitionMode`, `Algorithm`) jako pojedyncze bajty.

Napotkany błąd runtime. Po pierwszym udanym uruchomieniu pojawił się błąd `"Invalid IntervalSearchJoinExec, unsupported PartitionMode Auto in execute()"`. Przyczyna: sesja skonfigurowana pod `polars-bio` (`with_config_rt_bio()`) usuwa standardową regułę optymalizatora `join_selection`, która normalnie rozstrzyga `PartitionMode::Auto` na konkretny tryb — bez zastąpienia jej niczym innym. Naprawiono wymuszając w kodku `Auto` → `Partitioned` podczas deserializacji.

Wynik. 8/8 par, identyczne z baseline, na prawdziwym klastrze Ballista, z `algorithm: Coitrees` widocznym w planie fizycznym executora — pełna dystrybucja z **zachowanym algorytmem COITrees**, nie tylko z generycznym `HashJoinExec`.

Walidacja przenośności do polars-bio. Aby przenieść ten mechanizm z osobnego prototypu badawczego (`ballista_genomics/`) do samego `polars-bio` (Faza F), potrzeba: 1. Zaakceptowania (lub obejścia w inny sposób) jednoliniowej łatki widoczności — najlepiej przez zgłoszenie jej jako PR do `datafusion-bio-function-ranges` i oczekiwanie na wydanie z tą zmianą. 2. Ustandaryzowania miejsca rejestracji `LogicalExtensionCodec/PhysicalExtensionCodec` w momencie tworzenia sesji Ballista przez `polars-bio` (dziś to ręczna konfiguracja `SessionConfig` w prototypie). 3. Rozwiązania problemu `PartitionMode::Auto` w sposób trwalszy niż punktowa łatka w kodku — docelowo przez przywrócenie/zastąpienie reguły `join_selection` w `with_config_rt_bio()`. 4. Rozszerzenia tego samego wzorca na pozostałe operacje (`merge/nearest/coverage/subtract`) — wymaga to jednak współpracy z autorami `datafusion-bio-function-ranges` ponieważ ich węzły `*Exec` są prywatne (patrz Faza C wyżej); to większy zakres pracy niż sama łatka widoczności.

Wniosek: przeniesienie do `polars-bio` jest wykonalne technicznie dla `overlap`, ale wymaga świadomej decyzji dotyczącej samej biblioteki (zarówno `polars-bio`, jak i częściowo `datafusion-bio-function-ranges` w tym ekosystemie) co do akceptacji zależności od zvendorowanej łatki do czasu jej wejścia upstream.

4.2 Część 2 — próba osiągnięcia prawdziwej dystrybucji Saila

Krok 1: analiza trybu local-cluster. `Sail` udostępnia tryb `SAIL_MODE=local-cluster`, sugerujący wieloprocesowe/rozproszone wykonanie z wieloma workerami. Analiza kodu źródłowego `Saila` (`crates/sail-execution/src/worker_manager/`) wykazała, że istnieją tylko dwie implementacje `WorkerManager`: `LocalWorkerManager` (`workery` jako aktorzy w jednym procesie, `ActorSystem::spawn`) — używana **zarówno** przez tryb `Local`, jak i `local-cluster` — oraz `KubernetesWorkerManager` (tworzy prawdziwe pody `Kubernetes`). Genuinie wieloprocesowa/wielomaszynowa dystrybucja wymaga więc trybu `KubernetesCluster`.

Potwierdzono to empirycznie, nie tylko przez czytanie źródeł: zrzut procesów systemowych w trakcie działania klastra `local-cluster` z dwoma workerami (`ps aux + ss -tlnp`) wykazał dokładnie **jeden proces Python** (jeden PID), do którego należały wszystkie cztery nasłuchujące porty — `driver`, `worker 1`, `worker 2` i serwer `Spark Connect`. Brak jakiegokolwiek forka/subprocesu. Dodatkowym, mimowolnym dowodem prawdziwej granicy serializacji

(nie tylko wywołania w tym samym wątku) był błąd `TypeError: cannot pickle '_thread.lock' object` przy próbie użycia `threading.Lock()` do synchronizacji dostępu do globalnego kontekstu `DataFusion` w `polars-bio` — `UDTF` jest serializowany przez `cloudpickle` do wysłania tam, gdzie faktycznie się wykonuje, więc obiektu locka nie da się przez tę granicę przenieść. Zastąpiono lock retry-loopem z losowym backoffem — rozwiązanie działa, ale ujawniło przy okazji rzadki (niedeterministyczny) błąd współbieżnego dostępu do globalnego, mutowalnego kontekstu `DataFusion` w samym `polars-bio` przy równoległym wykonaniu wielu partycji na współdzielonym procesie.

Wniosek Części 2, etap 1: tryb `local-cluster` daje jedynie pozór wielowęzłowości (osobne porty, osobne role w logach), ale fizycznie jest jednoprocessowy. Prawdziwa dystrybucja wymaga `KubernetesCluster`.

Krok 2: próba `KubernetesCluster` lokalnie (k3s). Ponieważ maszyna testowa (WSL2, 3.5GB RAM łącznie) już raz doprowadziła do zawieszenia całego środowiska przy zwykłej kompilacji Rusta, przed instalacją czegokolwiek zbadano realistyczne wymagania pamięciowe: `k3s` (lżejsza alternatywa niż `kind`) wymaga oficjalnie minimum 2 CPU + 2GB RAM dla pojedynczego węzła serwera — **bez uwzględnienia obciążenia**, i bez wymogu Dockera (własny wbudowany containerd). `kind` nie podaje jasnej minimalnej liczby dla zwykłego uruchomienia, ale architektonicznie jest cięższy (pełne węzły Kubernetesa uruchamiane jako kontenery Dockera).

Jako zabezpieczenie zaplanowano twardy limit pamięci przez `cgroup v2` (`memory.max=1500M`, `memory.swap.max=0`, wymuszający szybki, czysty OOM-kill zamiast powolnego zamulenia całego systemu przez swap) oraz zwiększono ogólny margines systemowy: dodatkowy plik wymiany +2GB i obniżenie `vm.swappiness` z 60 do 10 (kernel wcześniej sięga po reclaim zamiast agresywnego swapowania).

Napotkany problem. Instalacja `k3s` (`INSTALL_K3S_SKIP_START=true`) powiodła się, ale próba uruchomienia go przez `systemd` (`systemctl start k3s`) zakończyła się błędem `System has not been booted with systemd as init system`. Weryfikacja wykazała, że ta instancja WSL **nie ma włączonego systemd** jako PID 1 (`systemctl is-system-running` → `offline`, prawdziwy init to `/init`) — wcześniejsza, błędna diagnoza (oparta o samo istnienie binarki `systemctl`) została skorygowana. Naprawa właściwa (`systemd=true` w `/etc/wsl.conf` + restart WSL) została odrzucona jako zbyt ryzykowna — zabiłaby całą trwającą sesję roboczą.

Zamiast tego podjęto próbę uruchomienia `k3s server` jako zwykłego procesu, ręcznie umieszczonego w ograniczonej `cgroupie` (bez udziału `systemd`). Próba zapisu limitów (`memory.max`, `memory.swap.max`) do nowo utworzonej `cgroupy` zakończyła się jednak **odmową dostępu nawet dla roota** (`Permission denied`) — mimo że kontroler `memory` widnieje w `cgroup.subtree_control` rodzica. Przyczyna nie została ostatecznie zdiagnozowana, prawdopodobnie to specyficzna dla tej instancji WSL2 usterka delegacji hierarchii `cgroup v2` bez `systemd` zarządzającego nią centralnie.

Rezultat. Proces `k3s server` wystartował **bez żadnego nałożonego limitu**. W ciągu ok. 60 sekund, jeszcze przed osiągnięciem pełnej gotowości klastra (trwała jeszcze instalacja domyślnych dodatków: `coredns`, `metrics-server`, `local-path-provisioner` przez `klipper-helm`), dostępna pamięć systemu spadała z 2,1GB do 171MB. Proces został natychmiast i bezpiecznie zatrzymany (`k3s-killall.sh`), po czym system wrócił do normy (dostępne 2,0GB). Wykonano pełne sprzątnięcie instalacji (`k3s-uninstall.sh`).

Wniosek Części 2, etap 2: to nie jest tylko teoretyczne ryzyko wynikające z dokumentacji — to zmierzony, potwierdzony brak wykonalności na tej konkretnej maszynie: nawet zanim `k3s` zdążył w pełni wystartować, zużył więcej pamięci niż wynosił cały dostępny zapas, a mechanizm bezpiecznego ograniczenia zasobów (`cgroup v2`) okazał się niedostępny w tym środowisku WSL2 bez `systemd`.

Ogólny wniosek Części 2: żadna z dwóch dostępnych lokalnie ścieżek do prawdziwej,

wieloprocusowej/wielomaszynowej dystrybucji Saila nie jest dziś wykonalna na tej maszynie — local-cluster z powodów architektonicznych samego Saila (jednoprocesowość niezależnie od zasobów), KubernetesCluster/k3s z powodów zasobowych i środowiskowych tej konkretnej instalacji WSL2. Jest to rezultat negatywny, ale rygorystycznie zweryfikowany empirycznie (nie tylko z dokumentacji) po obu stronach — realny i wartościowy punkt porównawczy między dojrzałością architektur dystrybucji obu silników, wart odnotowania w pracy: Ballista osiąga pełną, zweryfikowaną dystrybucję na tej samej maszynie (Część 1), Sail — nie, ze zidentyfikowaną, udokumentowaną przyczyną.

4.3 Część 3 — pełna dystrybucja dla merge, subtract, nearest i coverage (Faza H)

Punkt wyjścia i korekta wcześniejszej oceny. Faza C zakończyła się wnioskiem, że pełna dystrybucja pozostanie osiągnięciem specyficznym dla overlap, bo pozostałe operacje budują prywatne, niestandardowe węzły *Exec bezpośrednio w scan(), więc „wymagałyby współpracy z autorami crate’a albo reimplementacji od zera”. **Ta ocena była nieaktualna** — powstała, zanim crate został zvendorowany. Ponowna analiza źródeł wykazała, że moduły (pub mod merge, pub mod nearest, ...) są publiczne; blokadą są same struktury, zadeklarowane bez pub i pozbawione jakichkolwiek konstruktorów i getterów. Wystarczyła więc łatka tej samej natury co Łatka 1: **38 słów pub i 5 re-eksportów, zero linii logiki** (vendor/PATCH.md, Łatka 2).

Nowy, maszynowo sprawdzalny dowód dystrybucji. Wcześniej „rozproszoność” potwierdzano pośrednio — brakiem błędu serializacji i obecnością ShuffleReaderExec w wydruku. Ballista implementuje jednak EXPLAIN ANALYZE tak, że zwraca sekcje =====SuccessfulStage[stage_id=N, partitions=M]===== z drzewem operatorów i metrykami per etap. Każde uruchomienie zrzuca to teraz do output/dist_<op>_explain.txt, a osobny pakiet testów (tests/test_ballista_distribution_ev 16 asercji) sprawdza strukturę planu automatycznie. Ten pakiet celowo nie importuje polars-bio, więc wykonuje się w 0,03 s zamiast 4,5 minuty.

Wyniki dla poszczególnych operacji.

Operacja	Wzorzec	Etapy	Dowód w planie
merge	hash-shuffle po kontigu	3	Hash([chrom@0], 4); MergeExec czyta z ShuffleReaderExec
subtract	dwustronny hash-shuffle	4	dwa Hash([chrom@0], 4); SubtractExec z dwoma ShuffleReaderExec
nearest	broadcast lewej tabeli	2	NearestExec na 2 partycjach; lewa tabela nieobecna jako skan
coverage	broadcast + węzeł-nośnik	2	DistCoverageExec: coverage=true, broadcast_rows=5

Test poprawności jako test dystrybucji. Dane wejściowe rozbito na dwa pliki na tabelę i podzielono celowo: nakładające się gene_A1=[100,200) i gene_A2=[150,300) leżą w **różnych** plikach, więc trafiają do różnych partycji źródłowych. Poprawny wynik [100,300) może powstać wyłącznie wtedy, gdy hash-shuffle po chrom faktycznie przeniósł wiersze między partycjami. Gdyby dystrybucja przestała działać, test zwróciłby dwa osobne interwały zamiast jednego — głośno i jednoznacznie. To zamienia „test poprawności” w „test, czy rozproszenie jest prawdziwe”.

Trzy problemy warte odnotowania.

1. `target_partitions == 1` **cicho likwiduje dystrybucję**. Przy tej wartości DataFusion w ogóle nie wstawia hash-repartycji, więc zapytanie liczy się poprawnie, ale w jednym etapie. Objawu brak; wykrywalne wyłącznie przez EXPLAIN ANALYZE. Stąd jawne `with_target_partitions(4)` we wspólnej konfiguracji sesji.
2. **Ballista usuwa z planu rozproszonego każdą repartycję inną niż hash**. RoundRobinBatch, wstawiany przez `CountOverlapsProvider::scan()`, znika — dlatego nasz provider dla coverage celowo go nie wstawia, żeby plan lokalny i rozproszony miały ten sam kształt.
3. **Coverage wymagał czegoś więcej niż łatki widoczności**. `CountOverlapsProvider::scan()` materializuje lewą tabelę, przenosi ją do konstruktora indeksu i **porzuca** — powstały węzeł nie przechowuje ani jej danych, ani nazw jej kolumn, ani flagi coverage. Rozwiązaniem jest `DistCoverageExec`: transparentny dekorator delegujący wszystko do węzła vendora, ale przenoszący dodatkowo to, co tamten gubi.

Wynik negatywny: cluster **pozostaje niewykonalny**. `ClusterIdCoordinator` to bariera rendez-vous z `Vec<Waker>` w `Mutex`, działająca wyłącznie w obrębie jednego procesu — czeka, aż zgłoszą się **wszystkie** partycje, i dopiero wtedy liczy globalne przesunięcia identyfikatorów. Po rozproszeniu każdy executor dostałby własną kopię koordynatora, więc plan albo zawisłby w oczekiwaniu na partycje, które nigdy się nie zarejestrują, albo cicho zduplikowałby ID klastrów. To bloker **semantyczny**, którego żadna łatka widoczności nie usuwa. Wniosek wart zapisania w pracy: granica dystrybucji przebiega nie po widoczności API, lecz po tym, czy algorytm zakłada współdzieloną pamięć.

Skalowalność podejścia broadcast. Dla nearest i coverage lewa tabela jedzie w całości w ładunku planu fizycznego, który Ballista przesyła przez gRPC z limitem **16 MB**. To twarda granica tego wzorca i należy ją raportować jako właściwość rozwiązania, a nie obchodzić.

4.4 Część 4 — Sail odzyskuje równoległość; korekta diagnozy z Fazy B

Wszystkie pięć operacji działało już w Sailu, ale każda wymagała `.repartition(1)` przed wywołaniem UDTF-a. To obejście dawało poprawność kosztem **całej** równoległości — Sail liczył wszystko w jednej partycji, więc benchmarki nie mogłyby pokazać żadnego przyspieszenia ze skalowania.

Serię eksperymentów przeprowadzono na operacji `merge`, za każdym razem **bez** `.repartition(1)`, z wielokrotnymi powtórzeniami (objaw był niedeterministyczny, więc pojedyncze przejście niczego by nie dowodziło):

Wariant	Poprawnych
UDTF czysto pythonowy (merge napisany ręcznie, zero polars-bio)	3/3
UDTF → <code>pb.merge()</code> bez resetu kontekstu	1/3
UDTF → <code>pb.merge()</code> z <code>_reset_pb_context()</code>	0/3
<code>applyInPandas</code> → <code>pb.merge()</code> (zupełnie inna ścieżka kodowa Saila)	0/3
UDTF → <code>pb.merge()</code> przez lock w importowalnym module	5/5

Wniosek odwraca wcześniejszą atrybucję. Ten sam kształt zapytania — LATERAL nad UDTF-em nad tabelą na wielu partycjach — z funkcją czysto pythonową jest w 100% poprawny. Dodatkowo `applyInPandas`, całkowicie odrębna ścieżka kodowa, gubi grupy tak samo, co wyklucza wyjaśnienie specyficzne dla LATERAL. **W Sailu nie ma tu błędu.**

Prawdziwą przyczyną jest **globalny, mutowalny kontekst DataFusion w polars-bio**: gdy kilka partycji wykonuje `pb.*()` współbieżnie w jednym procesie, wywołania nadpisują sobie nawzajem zarejestrowane tabele (`s1/s2`). Co gorsza, pomocnik `_reset_pb_context()`, który

wprowadziliśmy sami we wcześniejszych fazach, pogarszał sytuację — jawnie deregistrował tabele, zamieniając wyścig w błąd deterministyczny (0/3 zamiast 1/3).

Rozwiązanie: moduł `sail_pb_guard` z lockiem serializującym dostęp do `polars-bio`. Lock musi mieszkać w **osobnym, importowalnym module**: obiektu `threading.Lock` nie da się umieścić w domknięciu UDTF-a, bo `cloudpickle` go nie zserializuje (`cannot pickle '_thread.lock' object`), natomiast moduł importowany po nazwie serializuje się przez **referencję**, więc wszystkie partycje w procesie sięgają po ten sam obiekt.

Znaczenie dla pracy jest potrójne: z Saila zdjęta zostaje niesłuszna krytyka; Sail odzyskuje realną równoległość, co ma bezpośrednie znaczenie dla przyszłych benchmarków; a przy okazji zidentyfikowane zostaje **realne i usuwalne ograniczenie polars-bio** — istotne tym bardziej, że dotyczy to bezpośrednio tej biblioteki.

5. Stan względem założeń projektu

Założenie	Status
Mechanizm UDF/UDTF jako runtime extension, bez forka silnika	TAK: Zrealizowane dla obu silników (Ballista: <code>LogicalExtensionCodec/PhysicalExtensionCodec</code> ; Sail: UDTF z PR #1519). Żaden z silników nie został sforkowany — jedyne modyfikacje to łatki widoczności w pomocniczej bibliotece
Porównanie dwóch silników, pełna implementacja w obu	TAK: Zrealizowane dla wszystkich 5 operacji z pierwotnego zakresu; po Fazie H wszystkie pięć działa w pełni rozproszone w Ballistrze
Wydajna implementacja + dobór algorytmów + benchmarki	CZĘŚCIOWO: Dobór algorytmu (<code>COITrees</code>) potwierdzony w pełni rozproszonej ścieżce Ballisty; benchmarki liczbowe — Faza D, nierozpoczęta
Zmiany docelowo wewnątrz <code>polars-bio</code>	ODŁOŻONE: Świadomie odłożone (Faza F, opcjonalna), wymaga osobnej decyzji dotyczącej biblioteki
Zmiana planu zapytania (repartycja wg chromosomu) w obu silnikach	TAK: Zweryfikowane w obu, maszynowo. Ballista: <code>partitioning=Hash([chrom@0], 4)</code> w <code>EXPLAIN ANALYZE (merge, subtract)</code> . Sail: <code>groupBy("chrom")</code> , od Fazy H bez wymuszania jednej partycji
UDF/UDTF faktycznie woła <code>polars-bio</code> , nie reimplementuje algorytmu	TAK: Potwierdzone w obu (Ballista woła <code>datafusion-bio-function-ranges</code> — silnik pod <code>polars-bio</code> ; Sail UDTF woła <code>pb.overlap()</code> wprost)

6. Znaleźiska i różnice semantyczne między silnikami/bibliotekami

- `nearest` — **KOREKTA (1 października 2026, plan 2)**: pierwotnie opisano tu różnicę w rozstrzyganiu remisów między natywnym `nearest()` z `datafusion-bio-function-ranges` a `pb.nearest()`. Była to pomyłka wynikająca z odwróconej orientacji: natywny `nearest()` zwraca wiersz na każdy wiersz PRAWEJ tabeli, a `pb.nearest(a, b)` — na każdy wiersz

a (ta sama odwrócona konwencja co w `coverage`). Przy poprawnej orientacji oba silniki wybierają na danych testowych tych samych sąsiadów, także przy remisie; szczegóły w `ballista_genomics/OPIS.md` („Plan 2 → Znaleziska”).

- `coverage`: `pb.coverage(a, b)` i natywne SQL-owe `coverage('reads', 'targets', ...)` mają odwróconą konwencję argumentów — to rzeczywista różnica API między bibliotekami, nie błąd.
- **Sail/pysail 0.5.3**: brak wsparcia dla argumentów `TABLE` w `UDTF`; `SparkSession.getOrCreate()` jako proces-globalny singleton powodujący błędy „Connection refused” po wielokrotnym tworzeniu serwera w jednym procesie (naprawione globalnie przejściem na `.create()`); `@udtf` sprawdza tryb (lokalny/zdalny) w momencie definicji klasy, nie rejestracji — klasy trzeba budować przez funkcje fabrykujące wywoływane po utworzeniu sesji.
- **polars-bio — najistotniejsze znaleźisko dla samej biblioteki**: globalny, mutowalny kontekst `DataFusion` nie jest bezpieczny przy współbieżnym dostępie z wielu wątków w tym samym procesie. Wywołania `pb.*()` z różnych partycji nadpisują sobie zarejestrowane tabele (`s1/s2`), przez co część grup cicho gubi wynik. To właśnie temu — a nie żadnemu błędowi Saila — przypisać należy objaw, który przez wcześniejsze fazy wymuszał obejście `.repartition(1)` (sekcja 4.4). Ograniczenie jest usuwalne bez modyfikowania `polars-bio` (lock po stronie wywołującego, `sail_pb_guard`), ale docelowo warto je usunąć w samej bibliotece.
- **Ballista — dwa zachowania, które cicho zmieniają plan rozproszony**: (1) przy `target_partitions == 1` hash-repartycja nie jest wstawiana w ogóle, więc zapytanie liczy się poprawnie, lecz w jednym etapie — bez objawu, wykrywalne tylko przez `EXPLAIN ANALYZE`;
(2) z planu rozproszonego usuwana jest każda repartycja inna niż hash, więc `RoundRobinBatch` wstawiony przez providera znika i nie daje żadnej równoległości.
- **Granica dystrybucji przebiega po założeniach algorytmu, nie po widoczności API**: `cluster` jest jedyną operacją, której nie da się rozproszyć — jego koordynator identyfikatorów to bariera `rendez-vous` działająca wyłącznie w obrębie jednego procesu. Żadna zmiana widoczności tego nie naprawi (sekcja 4.3).

7. Ograniczenia środowiskowe

Cała praca prowadzona jest na maszynie WSL2 z **3,5GB RAM łącznie**. Wymusiło to szereg praktyk metodologicznych: kompilacja Rusta wyłącznie z `CARGO_BUILD_JOBS=1` (domyślna, równoległa kompilacja raz doprowadziła do zawieszenia całego środowiska), `debug = false` w profilu kompilacji (unikanie wielogodzinnego narzutu linkera), nieuruchamianie ciężkich procesów Rusta i Pythona równolegle, oraz — jak opisano w sekcji 4.2 — bezpośredni wpływ na wykonalność eksperymentu z Kubernetesem/k3s.

8. Przeniesienie prac na GCP

Rozdział odpowiada wprost na pytanie postawione przy planowaniu tej fazy: czy do pracy z Google Cloud potrzebne jest jakieś IDE od Google, czy można zostać przy VS Code.

8.1 Nie, żadne IDE od Google nie jest potrzebne

Dostępne są cztery drogi; wszystkie pozwalają zostać przy VS Code:

1. **VS Code lokalnie + gcloud z terminala** — praca dokładnie jak dotąd w WSL, wdrożenie komendą. Wystarczy do wszystkiego, co przygotowano w katalogu `deploy/`.
2. **Rozszerzenie Cloud Code** — oficjalne, darmowe rozszerzenie Google do VS Code; wciąga do IDE obsługę GKE, Skaffold, kubectl i uwierzytelnianie. Przydatne przy pracy z Kubernetesem (czyli przy Sailu). Dalej jest to zwykły VS Code.
3. **VS Code Remote-SSH do maszyny GCE — rekomendowane dla tego projektu.** Ten sam model pracy co dziś z WSL, tylko „maszyna” stoi w chmurze. Rozwiązuje przy okazji ograniczenie 3,5 GB RAM: na `e2-standard-4` (16 GB) kompilacja Rusta może iść równolegle zamiast `CARGO_BUILD_JOBS=1`, a kilkudziesięciominutowe buildy schodzą do kilku minut.
4. **Cloud Shell / Cloud Workstations** — środowiska hostowane przez Google, dostępne z przeglądarki. Do tego projektu zbędne; wymienione dla kompletności.

Pułapka specyficzna dla WSL, warta odnotowania z góry: `gcloud` uruchomiony w WSL zapisuje klucze SSH do systemu plików WSL (`~/.ssh/google_compute_engine`), natomiast rozszerzenie Remote-SSH w wersji VS Code dla Windows czyta `C:\Users\<user>\.ssh\`. Jeśli VS Code nie widzi hosta, klucze trzeba skopiować — albo uruchamiać VS Code z poziomu WSL.

8.2 Dlaczego dwa różne modele wdrożenia

Podział nie wynika z wygody, lecz z architektury obu silników:

	Ballista	Sail
Model	scheduler + executory	driver + workery
Co wystarczy	maszyny GCE + Docker Compose	wymagany Kubernetes (GKE)
Dlaczego	Ballista ma udokumentowane wdrożenia Docker / Docker Compose / Kubernetes	Sail ma tylko dwie implementacje WorkerManager: LocalWorkerManager (workery jako aktory w jednym procesie — używana zarówno przez tryb <code>local</code> , jak i <code>local-cluster</code>) oraz <code>KubernetesWorkerManager</code>

Ustalenie dotyczące Saila jest potwierdzone empirycznie (sekcja 4.2), nie tylko z lektury źródeł. GKE jest zatem bezpośrednim rozwiązaniem problemu, który lokalnie okazał się nie do przejścia przy 3,5 GB RAM.

8.3 Przygotowane artefakty

Katalog `deploy/` zawiera gotowy punkt startu — **nic nie zostało jeszcze uruchomione w chmurze**:

- `deploy/ballista/` — `Dockerfile` (kompilacja dwuetapowa) i `docker-compose.yml` uruchamiający scheduler i dwa executory jako osobne kontenery. To krok pośredni między trybem `standalone` a GCP: łapie błędy pakowania i konfiguracji sieci lokalnie, czyli tanio.
- `deploy/sail/` — obraz z `pysail`, `polars-bio` i naszymi UDTF-ami (wraz z `sail_pb_guard`, bez którego współbieżne partycje gubią wyniki) oraz manifesty dla trybu `KubernetesCluster`. Workery celowo nie mają własnego manifestu: tworzy je sam sterownik przez API Kubernetesa, stąd potrzebne uprawnienia RBAC do zarządzania podami.

- `deploy/gcp/` — skrypty `gcloud`: konfiguracja projektu i bucketa, maszyna GCE dla Ballisty, klaster GKE dla Saila, oraz przewodnik po połączeniu VS Code z GCP.

8.4 Dane

`polars-bio` czyta `gs://` przez `OpenDAL`, więc pliki `BED/VCF` wgrywa się raz do bucketa GCS i podaje ścieżkę `gs://...` zamiast lokalnej — bez kopiowania czegokolwiek na maszynę.

8.5 Koszty i jak ich nie przepalić

Zasób	Koszt orientacyjny (sierpień 2026, europe-central2)
e2-medium (2 vCPU / 4 GB)	ok. 0,055 USD/h on-demand; ok. 0,033 USD/h spot
e2-standard-4 (4 vCPU / 16 GB)	ok. 0,15 USD/h on-demand
Warstwa sterowania GKE	0,10 USD/h za klaster, ale pierwszy klaster zonalny darmowy (kredyt ok. 74,40 USD/mies.)
GCS	ok. 0,02 USD za GB/mies. — przy danych testowych pomijalne
Nowe konto	300 USD kredytów na 90 dni

Ponieważ pierwszy darmowy klaster GKE musi być **zonalny**, przygotowany skrypt świadomie używa `--zone`, a nie `--region` — klaster regionalny tego kredytu nie otrzymuje.

Dwa nawyki obniżają rachunek najbardziej: zatrzymywanie maszyn po pracy (płaci się za czas działania, nie za samo istnienie) oraz używanie maszyn spot do benchmarków (60–70% taniej; ryzyko wywłaszczenia jest przy powtarzalnych testach akceptowalne). Warto też od razu ustawić budżet z alertem mailowym.

9. Otwarte kroki i dalsze fazy

- **Faza D (benchmarking i GCP)** — nierozpoczęta, ale przygotowana: artefakty wdrożeniowe są gotowe (sekcja 8.3). Pozostaje ustalenie dostępu do budżetu/projektu GCP oraz właściwe pomiary przy skalowaniu liczby węzłów i rozmiaru danych. Dopiero teraz mają one sens po obu stronach: Ballista rozprasza cztery operacje z realnym shuffle, a Sail odzyskał równoległość po zdjęciu obejścia `.repartition(1)`.
- **Klaster Ballista jako osobne procesy** — dotychczasowe wyniki pochodzą z trybu `standalone` (`scheduler` i `executor` w jednym procesie). Plan jest tam realnie serializowany i przechodzi przez shuffle, ale nie przez prawdziwą sieć. `deploy/ballista/docker-compose.yml` jest przygotowany; wymaga drobnej zmiany w kodzie: `remote_with_state()` zamiast `standalone_with_state()`.
- **Zgłoszenie łatek widoczności upstream** do `biodatageeks/datafusion-bio-functions` — obie są czystymi zmianami widoczności, gotowymi jako jeden mały PR. Po ich przyjęciu katalog `vendor/` przestałby być potrzebny.
- **Usunięcie ograniczenia współbieżności w polars-bio** (sekcja 4.4) — do rozważenia razem na poziomie samej biblioteki.
- **Synchronizacja i domknięcie zakresu** — aktualizacja `architektura_draft.md` wynikami dotychczasowych prac; pytania nadal otwarte: natywny distributed Polars jako trzecia ścieżka porównawcza? Dostępność zasobów GCP? Kwestia zakresu `cluster/complement` (poza pierwotną piątką operacji).

- **Faza F (zmiany wewnątrz polars-bio, opcjonalna)** — odłożona do czasu potwierdzenia wykonalności w Fazach A-D i osobnej decyzji dotyczącej biblioteki, zwłaszcza w kontekście zależności od zvendorowanej łątki widoczności opisanej w sekcji 4.1.
- Klaster Ballista jako osobne procesy systemu operacyjnego (nie in-proc) — krok wart wykonania przed Fazą D, żeby przetestować rzeczywistą sieć, nie tylko API in-proc.
- Zgłoszenie łątki widoczności (`vendor/PATCH.md`) jako PR upstream do `biodatageeks/datafusion-bio-functions`